

Facultad Ingeniería de Ciencias y Sistemas

Organización de Lenguajes y Compiladores 2

Sección A

Matthew Emmanuel Reyes Melgar

202202233

MANUAL TÉCNICO

Guatemala 19 de Marzo del año 2025

INTRODUCCIÓN

El presente documento describe el desarrollo de un compilador para el lenguaje Go, implementado mediante una API en .NET y un frontend con React Vite. Este proyecto tiene como objetivo principal proporcionar una base sólida para la construcción de un compilador funcional, aunque no implemente todas las capacidades del lenguaje Go, sí establece una estructura adecuada para su expansión y mejora.

Para el procesamiento del lenguaje se utilizó ANTLR4, una herramienta potente para el reconocimiento y procesamiento de gramáticas. Se implementó el enfoque Visitor, lo que permitió una mejor organización del código y una separación clara de los componentes gramaticales mediante etiquetas (#). Esto facilitó la estructuración de las reglas de análisis léxico, sintáctico y semántico, permitiendo una mayor modularidad y mantenimiento del código.

Además del análisis y compilación del código fuente, la API está diseñada para generar reportes de errores léxicos, sintácticos y semánticos. También ofrece funcionalidades para la creación de reportes de tabla de símbolos, proporcionando información detallada sobre los elementos identificados durante la ejecución del código.

Este proyecto representa un primer paso hacia el desarrollo de un compilador completo para Go, con una arquitectura flexible y escalable que permitirá futuras mejoras y ampliaciones en su funcionalidad.

REQUERIMIENTOS

Para poder ejecutar correctamente el compilador desarrollado, es necesario contar con una serie de herramientas y configuraciones previas en el entorno de desarrollo. A continuación, se detallan los requerimientos esenciales:

1. Instalación de .NET 9.0

El backend de la aplicación está desarrollado en **.NET 9.0**, por lo que es fundamental tener instalada esta versión o una compatible para ejecutar la API. Para instalar .NET 9.0, se recomienda seguir los siguientes pasos:

- Descargar el SDK de .NET desde el sitio oficial de Microsoft:
<https://dotnet.microsoft.com/en-us/download>
- Seguir las instrucciones de instalación según el sistema operativo.
- Verificar la instalación ejecutando el siguiente comando en la terminal o consola:
`dotnet --version`
- Esto debería devolver la versión instalada de .NET.

2. Instalación de ANTLR4

ANTLR4 es una herramienta esencial para la construcción del compilador, ya que permite definir y procesar la gramática del lenguaje Go. Para instalar ANTLR4, se deben seguir estos pasos:

- Descargar ANTLR4 desde el sitio oficial: <https://www.antlr.org/>
- Agregar ANTLR4 a la variable de entorno para que pueda ser utilizado desde la terminal: `export CLASSPATH=".:usr/local/lib/antlr-4.9.3-complete.jar:$CLASSPATH"`
- `alias antlr4='java -jar /usr/local/lib/antlr-4.9.3-complete.jar'`
- `alias grun='java org.antlr.v4.gui.TestRig'`
- *(Los comandos pueden variar según el sistema operativo. Para Windows, se recomienda agregar el archivo JAR a las variables de entorno y usar `java -jar` directamente.)*
- Para generar los archivos necesarios a partir de la gramática definida, se usa el siguiente comando: `antlr4 -Dlanguage=CSharp -visitor -no-listener MyGrammar.g4`
-

3. Instalación de Node.js y React Vite

El frontend del proyecto fue desarrollado utilizando **React con Vite**, por lo que se requiere tener instalado **Node.js**. Para la instalación:

- Descargar e instalar Node.js desde: <https://nodejs.org/>
- Verificar la instalación ejecutando: `node -v`
- `npm -v`
- Esto debe devolver la versión instalada de Node.js y npm.
- Para crear un nuevo proyecto con Vite o ejecutar el existente, se puede utilizar el siguiente comando: `npm create vite@latest my-project --template react`
- Luego, dentro del proyecto: `npm install`
- `npm run dev`

- Esto iniciará el servidor de desarrollo en el navegador.

4. Clonación y Ejecución del Proyecto

Una vez instaladas las dependencias anteriores, es necesario clonar el repositorio del proyecto y ejecutar la API y el frontend:

Backend:

```
cd OLCAPI
```

```
dotnet restore
```

```
dotnet run
```

Frontend:

```
cd OLCCLIENT
```

```
npm install
```

```
npm run dev
```

Con estos pasos, se tendrá el compilador funcionando correctamente y listo para procesar código en Go con ANTLR4, la API en .NET y la interfaz en React con Vite.

FRONT END – APP:JSX

Funciones del Código en App.jsx

Este código define una aplicación en React que proporciona un editor de código y una consola de salida, usando la biblioteca **CodeMirror** para mejorar la experiencia de edición. Se incluyen funcionalidades para **abrir archivos**, **ejecutar código** y **limpiar el editor**.

1. *openFile()* - Función para abrir archivos

- **Objetivo:** Permite al usuario seleccionar y cargar un archivo de texto en el editor de código.
- **Proceso:**
 1. Crea dinámicamente un elemento `<input type="file">`.
 2. Al seleccionar un archivo, lo lee usando `FileReader`.
 3. Una vez leído, su contenido se carga en el editor `CodeMirror`.

```
const openFile = () => {  
  
  var input = document.createElement('input');  
  
  input.type = 'file';  
  
  input.onchange = e => {  
  
    var file = e.target.files[0];  
  
    var reader = new FileReader();  
  
    reader.readAsText(file, 'UTF-8');  
  
    reader.onload = readerEvent => {  
  
      var content = readerEvent.target.result;  
  
      editorRef.current.setValue(content);  
  
    };  
  
  };  
  
  input.click();  
  
};
```

2. *runCode()* - Función para ejecutar código

- **Objetivo:** Envía el código del editor a un servidor para su compilación y muestra la salida en la consola.
- **Proceso:**
 1. Obtiene el código del editor usando `editorRef.current.getValue()`.
 2. Lo envía a un servidor en `http://localhost:5031/compile` mediante una solicitud **POST**.
 3. Si el servidor responde con errores, se analizan y se muestran en la consola.
 4. Si la ejecución es correcta, muestra el resultado en la consola.

```
const runCode = async () => {

  const code = editorRef.current.getValue();

  let output = "";

  try {

    const response = await fetch('http://localhost:5031/compile', {

      method: 'POST',

      headers: {

        'Content-Type': 'application/json',

      },

      body: JSON.stringify({ code }),

    });

    if (!response.ok) {

      let errorText = await response.text();

      try {

        const errorJson = JSON.parse(errorText);

        if (errorJson.errores_lexicos || errorJson.errores_sintacticos) {

          if (errorJson.errores_lexicos.length > 0) {

            output += errorJson.errores_lexicos.join('\n') + '\n';

          }

          if (errorJson.errores_sintacticos.length > 0) {

            output += errorJson.errores_sintacticos.join('\n') + '\n';

          }

        } else {
```

```

        output += `${errorText}\n`;
    }
} catch {
    output += `Error del servidor: ${errorText}\n`;
}
} else {
    const data = await response.json();

    if (data.errores_lexicos.length > 0) {
        output += 'Errores léxicos:\n' + data.errores_lexicos.join('\n') + '\n';
    }

    if (data.errores_sintacticos.length > 0) {
        output += 'Errores sintácticos:\n' + data.errores_sintacticos.join('\n') + '\n';
    }

    if (data.result) {
        output += `Resultado:\n${data.result}\n`;
    } else if (!data.errores_lexicos && !data.errores_sintacticos) {
        output += 'No hay salida.';
    }
}

} catch (error) {
    output += 'Error al conectar con el servidor.\n';
    console.error('Error:', error);
}

consoleEditorRef.current.setValue(output);
};

```

3. *clearCode()* - Función para limpiar el editor y la consola

- **Objetivo:** Borra el contenido del editor y la consola.
- **Proceso:**
 1. Llama a `setValue("")` en `editorRef.current` y `consoleEditorRef.current` para limpiar ambos campos.

```
const clearCode = () => {  
  
  editorRef.current.setValue("");  
  
  consoleEditorRef.current.setValue("");  
  
};
```

Interacciones con los Botones

Estos métodos se asocian a los botones de la interfaz gráfica dentro de `useEffect()`. Se añaden `addEventListener` al montar el componente y se eliminan al desmontarlo para evitar fugas de memoria.

```
openButton.addEventListener('click', openFile);  
  
runButton.addEventListener('click', runCode);  
  
clearButton.addEventListener('click', clearCode);
```

Al desmontar el componente, se eliminan los eventos:

```
return () => {  
  
  openButton.removeEventListener('click', openFile);  
  
  runButton.removeEventListener('click', runCode);  
  
  clearButton.removeEventListener('click', clearCode);  
  
};
```

CÓDIGO BACKEND

Este código es un **controlador de backend** en **ASP.NET Core** que proporciona un endpoint POST /compile para compilar código enviado por el cliente. Usa **ANTLR4** para analizar el código y detectar errores léxicos y sintácticos antes de procesarlo.

Explicación del código:

Estructura general

- **Framework:** ASP.NET Core
- **Namespace:** api.Controllers
- **Clase principal:** Compile (hereda de Controller)
- **Ruta del endpoint:** [Route("[controller]")] → Se expone como /compile
- **Método principal:** Post([FromBody] CompileRequest request)

Componentes clave

1. Clase CompileRequest

Define la estructura del JSON que recibirá el servidor en la petición POST.

```
public class CompileRequest
{
    [Required]
    public required string code { get; set; }
}
```

Propiedad:

- code → Representa el código fuente que se enviará para analizar y compilar.

Validación:

- [Required] → Asegura que el campo code no esté vacío en la petición.

2. Método Post (Recibe el código y lo analiza)

Este método maneja la solicitud POST /compile y sigue estos pasos:

Valida la entrada:

- Si el code es inválido, retorna un error 400 Bad Request.

Usa ANTLR para analizar el código fuente:

- Convierte el código en un AntlrInputStream.
- Usa gramaticaLexer y gramaticaParser para generar el árbol de sintaxis.
- Agrega **listeners** personalizados para capturar errores léxicos y sintácticos.

Verifica errores de análisis:

- Si hay **errores léxicos o sintácticos**, retorna 400 Bad Request con los errores detectados.

Ejecuta la compilación:

- Usa SearchTree para recorrer el árbol sintáctico.
- Usa CompilerVisitor para visitar nodos específicos (declaraciones, funciones, estructuras, asignaciones, etc.).
- Genera una tabla de símbolos con SymbolTable.GenerateHtmlReport(tableSymbol).

Retorna los resultados:

- Si la compilación es exitosa, responde con 200 OK y el resultado.
- Si hay errores semánticos o de análisis, devuelve un mensaje de error con 400 Bad Request.
- Si ocurre un error inesperado, devuelve 500 Internal Server Error.

Explicación detallada del proceso de compilación

```
var inputStream = new AntlrInputStream(request.code);
```

```
var lexer = new gramaticaLexer(inputStream);
```

```
var tokens = new CommonTokenStream(lexer);
```

```
var parser = new gramaticaParser(tokens);
```

- Convierte el código recibido (request.code) en un flujo de entrada para ANTLR.
- **Lexer (gramaticaLexer)**: Divide el código en tokens.
- **Parser (gramaticaParser)**: Analiza la estructura sintáctica y crea un árbol de análisis.

Manejo de errores léxicos y sintácticos

```
var lexicalErrorListener = new LexicalErrorListener();
```

```
var syntaxErrorListener = new SyntaxErrorListener();
```

```
lexer.RemoveErrorListeners();
```

```
lexer.AddErrorListener(lexicalErrorListener);
```

```
parser.RemoveErrorListeners();
```

```
parser.AddErrorListener(syntaxErrorListener);
```

Función:

- Reemplaza los manejadores de error predeterminados de ANTLR con **listeners personalizados** (LexicalErrorListener y SyntaxErrorListener).
- Captura errores **léxicos** (tokens mal formados) y **sintácticos** (errores de estructura).

Ejecución del código compilado

```
SearchTree searchTree = new SearchTree();
```

```
searchTree.Visit(tree);
```

```
var visitor = new CompilerVisitor();
```

Función:

- SearchTree **recorre el árbol de análisis** y extrae información relevante del código.
- CompilerVisitor **procesa las declaraciones, asignaciones y estructuras del código fuente**.

```
foreach (var dclSimple in searchTree.declaracionesSimples)
```

```
{
```

```
    visitor.Visit(dclSimple);
```

```
}
```

Función:

- Visita **las declaraciones** en el código analizado para procesarlas.

Manejo de errores en la compilación

```
catch (ParseCanceledException ex)
```

```
{
```

```
    return BadRequest(new { error = "Error en el análisis del código: " + ex.Message });
```

```
}
```

```
catch (ErrorSemantico ex)
```

```
{
```

```
    return BadRequest(new { error = ex.Message });
```

```
}
```

```
catch (Exception ex)
```

```
{
```

```
    _logger.LogError(ex, "Error inesperado en la compilación.");
```

```
return StatusCode(500, new { error = "Error interno del servidor." });  
}
```

Función:

- **Errores de análisis (ParseCanceledException)** → Devuelve 400 Bad Request.
- **Errores semánticos (ErrorSemantico)** → Devuelve 400 Bad Request con el mensaje del error.
- **Errores inesperados (Exception)** → Devuelve 500 Internal Server Error.